Dialog with the AI assistant

Assistant: Claude (Anthropic), in the claude.ai app, working in a cloud workspace with file, shell, and web tools.

Dates: October 20–24, 2026 (times are US Central).

Format: every message exchanged in the chat, in order. The user's messages are reproduced as sent; screenshots are described in brackets. The assistant's replies are reproduced as written. Its short progress notes between steps are included as written, marked *(progress note)*. The assistant's tool use (commands, file edits, searches) is not shown; its effects are the files in this repository and its Git history.

Part 1 — Building the project (October 20)

User — 8:26 PM

[Screenshot of the assignment: "Management Problem" (running Python-built deep learning models in production with Go via the Open Neural Network Exchange, with a figure of ONNX connecting GoMLX, R onnx, Rust ort, TensorFlow/Keras, PyTorch, and Scikit-Learn), "Assignment Requirements" (use free AI tools; develop a model in TensorFlow/Keras, Scikit-Learn, or PyTorch; obtain Python predictions on a large test set and time them; export to ONNX; read it into GoMLX; show Go predictions match and time them; compare times in README.md), "Grading Guidelines (100 Total Points)", and "Assignment Deliverables".]

Assistant

> I can build this whole project for you: a Python model, the ONNX export, a Go program that runs it with GoMLX, tests, benchmarks and the README. A few choices up front decide what I build.

The assistant asked three multiple-choice questions. The user answered:

- **Which model should the project use?** — "You pick one (Recommended)": a PyTorch neural-net classifier on a built-in dataset with a large test set.
- **Which operating system should the build-an-executable instructions target?** — "Windows (.exe)"
- **How do you want to receive the project?** — "Push to my GitHub repo"

Assistant

> Got it — PyTorch model, Windows instructions, pushed to your GitHub. What's the repo's **owner/name** (e.g. `yourname/onnx-gomlx-benchmark`)? It can be a new empty repo. I'll start building in the meantime.

> *(progress note)* Fashion-MNIST is available (10,000-image test set), so I'll use a small PyTorch CNN on it. Checking which ONNX ops GoMLX supports:

User — 8:58 PM

> CHbobray/onnx-gomlx-benchmark

Assistant

> *(progress note)* Now the config flag and the report changes for latency.

> Heads-up on pushing: GitHub reports that Claude can read **CHbobray/onnx-gomlx-benchmark** but isn't allowed to push to it yet. To fix that, install the Claude GitHub App on that repo at https://github.com/apps/claude/installations/select_target (or reconnect GitHub under claude.ai → Settings → Connectors). I'll keep building meanwhile; if access isn't there when I'm done, I'll send you a zip with the full repo plus the exact `git` commands to push it yourself.

> *(progress note)* Empty repo — trying the push now in case access has been granted.

Assistant — summary of the first version

> The full project is on GitHub at **https://github.com/CHbobray/onnx-gomlx-benchmark** — that's the URL to submit. The push went through, so the access warning from earlier didn't block it. A fresh copy of the repo builds, all Go and Python tests pass, and it produces a 14 MB Windows `.exe`.
>
> **What I built:** - **Model:** a small PyTorch image classifier for Fashion-MNIST (photos of clothing in 10 classes). It scores 90.3% on the 10,000-image test set. - **Go program:** it loads the ONNX file with GoMLX, checks every prediction against Python's, and times the run. - **Testing:** unit tests on both sides, a Monte Carlo benchmark, CPU and memory profiles, and a `build.ps1` script for Windows.
>
> **Results:** - **Predictions match exactly.** Go and Python agree on the class for all 10,000 images, and the largest difference in any output score is 0.0000076. - **Go is much slower.** GoMLX's pure-Go engine is about 20x slower than PyTorch and about 170x slower than ONNX Runtime run from Python. One image takes about 3.7 ms in

> Go versus about 0.2 ms in PyTorch. The profile shows nearly all the time goes to GoMLX's simple convolution and pooling code. - **Recommendation in the README:** the Python-to-Go workflow is proven. For a fast, high-traffic service, though, Go should call a native engine (GoMLX's XLA backend or ONNX Runtime) rather than the pure-Go one.
>
> **Things you need to do:** 1. **Add the AI dialog file.** Save this conversation as a PDF (open it in the browser, then Print → Save as PDF). Commit it to `docs/ai-assistant-dialog.pdf`, which is where the README points. 2. **Use Go 1.27 or newer on Windows.** The current onnx-gomlx library requires it. 3. **Optional: rerun on your machine.** The README's timing numbers come from my 2-core Linux sandbox. If your instructor wants numbers from your own computer, run steps 4 and 5 in the README and paste the tables from `results\timing_comparison.md` into it.
>
> I hit one compatibility problem worth knowing about, and the README covers it. PyTorch always writes a default setting (`dilations=[1,1]`) on its pooling layers, and GoMLX refuses to load any pooling layer that has it. The export script removes that setting only when it's the default, which doesn't change what the model computes, and a test confirms the model still matches PyTorch.

Part 2 — Checking against the GoMLX documentation (October 20)

User — 9:39 PM

> Does the output account for this? https://github.com/gomlx/gomlx

Assistant

> *(progress note)* I'll check the GoMLX README against what the project does.

> *(progress note)* Now the label fix and a test for the backend override:

> *(progress note)* Tests pass with the new backend test. Now the full 30-run benchmark on the ONNX Runtime backend:

> *(progress note)* The SIMD build works for Windows too, so I'll turn it on in `build.ps1` as GoMLX recommends. Now updating the README:

Assistant — summary of the backend review

> No, it didn't fully account for it. I checked the GoMLX README against the code and found three gaps. I fixed them, and the fixes are pushed to your repo.
>
> 1. **It ignored GoMLX's backend setting.** GoMLX lets you pick its compute engine (the "backend") with the `GOMLX_BACKEND` environment variable. My program always forced the pure-Go backend. It now reads that variable, and the results tables are labeled with the backend actually used.
> 2. **SIMD needs a build flag.** GoMLX's fast CPU code for AVX2/AVX-512 is only compiled in when you build with `GOEXPERIMENT=simd`, and my first build left it out. `build.ps1` now sets the flag. I re-measured, and it barely changed the result (29.9 s → 29.4 s for the test set). SIMD covers matrix math, but convolution and pooling take about 80% of the time and have no SIMD code yet. I also tried the README's `ops_parallel` setting, with no real gain.
> 3. **The README lists an ONNX Runtime backend I hadn't tried, and it changes the conclusion.** The same Go program on that backend:
>
> | | Test set (10,000 images) | One image |
> |---|---|---|
> | Go + GoMLX, ONNX Runtime backend | **223 ms** | **0.08 ms** |
> | Python + PyTorch | 1,506 ms | 0.18 ms |
> | Go + GoMLX, pure-Go backend | 29,893 ms | 3.69 ms |
>
>   Predictions still match Python on all 10,000 images. So Go is about 7x faster than PyTorch on this backend, versus 20x slower on pure Go.
>
> **The catch:** this fast backend doesn't work on Windows with the current versions. It needs a C compiler, and its loader is only written for Linux and macOS. The GoMLX README also lists the XLA backend for Linux and macOS only. So your Windows `.exe` still has to use the slower pure-Go backend. I added the ONNX Runtime backend as an optional Linux/macOS build (`go build -tags onnx`).
>
> **README changes:** - A "GoMLX backends tried" table covering every option above. - A revised recommendation: use GoMLX on the ONNX Runtime backend on Linux servers for the high-traffic service, and keep pure Go for Windows tools and anything that must ship as a single file.
>
> All tests still pass.

Part 3 — Checking against the grading rubric (October 20)

User — 9:55 PM

[Screenshot of the “Grading Guidelines (100 Total Points)” — coding rules, organization, and aesthetics; testing and software metrics; design and development; documentation; application — and the “Assignment Deliverables”.]

> Check these:

Assistant

> *(progress note)* I’ll audit the repo against each rubric item, fix what falls short, then report back.

> *(progress note)* Now restructuring `main.go`: `run()` split into one function per step, plus an automatic run log, progress lines, and a time estimate before long benchmarks.

> *(progress note)* The error message should still go to stderr and also be recorded in the log, so I’ll adjust that:

> *(progress note)* Next: per-component memory and CPU benchmarks (`go test -bench -benchmem`), so the metrics cover parts of the program as well as the whole run.

> *(progress note)* Progress lines, a time estimate, and the automatic run log (with start time, command, and outcome) all work. The Python scripts also print only to the screen, so they’ll get log files too:

> *(progress note)* Calling the logging setup at import time would overwrite the logs whenever pytest imports these modules, so I’ll move it into `main()`:

> *(progress note)* That check overwrote the real Python results with a 1-run test, so I’m restoring the committed files:

> *(progress note)* Restored. Next, the LLM dialog deliverable: a plain-text record of this conversation in `docs/`, which the assignment accepts.

> *(progress note)* Now updating the README for the logs, component metrics, progress output, and dialog file:

> *(progress note)* The `results/` logs were captured before the program wrote its own logs, so I'll note that rather than claim they have the new header:

> *(progress note)* `cmd/fashion-go` coverage is only 30% because `run()` itself is untested. I'll add an end-to-end test that runs the whole program on a 40-image slice of the real data:

> *(progress note)* Coverage for `cmd/fashion-go` is up from 30% to 72%. One test comment doesn't describe what the code does, so fixing it, then updating the README's test table:

Assistant — summary of the rubric review

> I checked the repo against each grading item and all five deliverables, and fixed every gap I found. The changes are pushed, all tests pass, and the Windows `.exe` still builds.
>
> **Coding rules (20)** — this needed work. The main `run()` function was 84 lines; it now reads as a short list of steps, each in its own function. I also tightened a few small things: a repeated type is now named, empty input is caught, and test helpers check their errors.
>
> **Testing and metrics (20)** — three gaps, all fixed: - **Logs:** the program never wrote its own log; I'd made the run logs by capturing screen output. The Go program now writes `results/go_run_log.txt` itself, with start time, exact command, and whether it finished or why it failed. The Python scripts now log with timestamps too. - **Component metrics:** the rubric asks for memory and processing use of components as well as the whole program. I added per-component benchmarks (time, memory, allocations) and a README table. Inference is about 2 MB of throwaway memory per image; everything else is negligible. - **Untested main program:** the code that drives a whole run had no tests. New end-to-end tests run the full program on 40 real images, including the "predictions don't match" path. Its coverage went from 30% to 72%; the other packages are at 88–92%.
>
> The Monte Carlo benchmark was already in place.
>
> **Design (20)** — already in good shape: reusable packages, and the code covers every assignment step.
>
> **Documentation (20)** — the README already explained each program and data file, plus testing and usage. I added the logs, component metrics, and progress output.
>
> **Application (20)** — one real UX problem. Double-clicking the `.exe` started about 17 minutes of benchmarking with no output. It now gives a time estimate, a tip for a shorter run, and a progress line per run with time remaining. Input checking and the Windows build instructions were already done.

> **Deliverables:** - **GitHub URL:** https://github.com/CHbobray/onnx-gomlx-benchmark - **LLM dialog file:** this was missing. I added `docs/ai-assistant-dialog.md`, every message in our chat as plain text, which the assignment accepts. It ends just before this reply, so for a complete record also save the chat as PDF (Print → Save as PDF) and commit it as `docs/ai-assistant-dialog.pdf`. - **README completeness:** done.
>
> **Two things I can't change from here:** - **Commit history:** the repo has 3 commits. If "effective use of Git" is judged on history, further work should go in small, clearly described commits. - **Timing numbers:** they come from my 2-core Linux sandbox. The `results/` logs are from that run, captured before the program wrote its own logs (the README says so). Rerunning on your Windows machine would give you your own numbers and logs in the new format.

Part 4 — Week 5 discussion post (October 21)

User — 8:02 AM

[Screenshot of the "Week 5. Generative AI and Go Programming" discussion prompt: review your experience using generative AI and large language models for writing code; compare tools (GitHub Copilot, OpenAI's Codex, Anthropic's Claude Code, Google's Gemini CLI, DeepSeek); consider whether AI assistants are better with Python or Go; and answer what tools you have tried, what works and what doesn't, and whether AI coding tools have helped you with Python and Go.]

> Draft a response in layman's terms like a person would talk at the college level

Assistant — first draft (shown in a draft card)

> This week I used Claude (Anthropic) for pretty much the whole assignment. I had it build a small PyTorch image classifier on Fashion-MNIST, export it to ONNX, load it into Go with GoMLX, and check that both languages gave the same predictions. They did, on all 10,000 test images. Getting that working was the impressive part. It wrote the Python, the Go, the unit tests, and the README, and the Go code compiled and was organized into clean, reusable packages.
>
> What didn't work as well: the first version looked finished but wasn't the full picture. When I pointed it to the GoMLX GitHub page, it found it had missed things. It had hard-coded the slowest backend, it skipped a build flag the docs say you need for the fast CPU code, and it never tried GoMLX's ONNX Runtime backend. That changed the whole conclusion. On the pure-Go backend, Go was about 20 times slower than PyTorch. On the ONNX Runtime backend, the same Go program was about 7 times faster than PyTorch. When I had it check the work against our grading rubric, it also found gaps: no real program logs, one function that was way too long, and a program that sat silent for about 17 minutes while it benchmarked. So my biggest takeaway is

> that AI is great at producing a lot of work fast, but you still have to check it against the documentation and the requirements yourself. It usually doesn't know what it doesn't know until you push it.
>
> On the Python vs. Go question, my experience was a little different from what I expected. The Python side was routine, probably because there is so much PyTorch code out there. The Go side was harder, but not because of Go itself. GoMLX is a newer, niche library that changes fast, so the AI had to read the library's source code to figure out how to use it. It also ran into a compatibility issue where PyTorch writes a setting that GoMLX refuses to load. Go's strictness did help, though. The compiler catches a lot of mistakes right away, so bad AI code fails loudly instead of quietly running wrong.
>
> Has it made me a better programmer? I think it made me a better reviewer more than a better coder. I spent more time asking whether the code was right and complete than typing it. Going forward, I'd like to try Gemini and DeepSeek on the same task to see if any of them handle Go libraries like GoMLX better.

> This post sticks to what happened in this project. If you've used other AI tools before, like ChatGPT or Copilot, add a sentence about them near the start, since the prompt asks what you've tried so far. Also change the last line if Gemini and DeepSeek aren't actually the tools you plan to try.

User — 8:48 AM

> I have used ChatGPT and Copilot. Please update and make more concise

Assistant — second draft (shown in a draft card)

> I've used ChatGPT and GitHub Copilot before, and this week I used Claude for the whole assignment. It built a PyTorch classifier, exported it to ONNX, and ran it in Go with GoMLX. Python and Go gave the same predictions on all 10,000 test images, and the Go code compiled cleanly into reusable packages.
>
> What didn't work: the first version looked finished but wasn't. When I pointed it to the GoMLX docs, it found it had used the slowest backend and never tried the ONNX Runtime one. That flipped the result. Go went from about 20x slower than PyTorch to about 7x faster. Checking against our rubric also turned up missing logs and an overly long function. My takeaway: AI produces a lot fast, but you still have to check it against the docs and requirements.
>
> On Python vs. Go, the Python side was routine, likely because there's so much PyTorch code out there. Go was harder, but because GoMLX is a newer niche library, not because of the language. Go's strict compiler actually helped, since bad AI code fails right away instead of quietly running wrong.

> Overall, it’s made me a better reviewer more than a better coder. Next I’d like to try Gemini and DeepSeek on the same task.

> The post is now about half as long and mentions ChatGPT and Copilot in the first line. I didn’t make up anything about how you used them. If you add one specific detail, like what you’ve used Copilot for, it will read more like your own experience and less like a summary.

User — 9:23 AM

> Make even more casue

Assistant — third draft (shown in a draft card)

> I’ve messed around with ChatGPT and GitHub Copilot before, but this week I leaned on Claude for basically the whole assignment. It built the PyTorch model, exported it to ONNX, and got it running in Go with GoMLX. Python and Go ended up with the exact same predictions on all 10,000 test images, which was pretty cool to see.
>
> The catch is the first version looked done but really wasn’t. Once I pointed it at the GoMLX docs, it realized it had been using the slowest backend and never even tried the ONNX Runtime one. That totally flipped the results. Go went from about 20x slower than PyTorch to about 7x faster. Running it against our rubric caught some other stuff too, like missing logs and a function that was way too long. So yeah, AI cranks out a ton of work fast, but you still have to double-check it.
>
> As for Python vs. Go, the Python part was easy, probably because there’s so much PyTorch code out there. Go was trickier, but honestly that was more about GoMLX being a newer, niche library than Go itself. Go’s strict compiler actually helped, because when the AI got something wrong, it broke right away instead of quietly giving bad results.
>
> Honestly, I think it’s made me a better code reviewer more than a better coder. Next I want to try Gemini and DeepSeek on the same thing and see how they compare.

> This version sounds more like a classmate talking, with phrases like “messed around with,” “pretty cool,” and “so yeah.” It’s about the same length as the last one.

Part 5 — This file (October 24)

User — 1:57 PM

> Generate this file: docs/ai-assistant-dialog.pdf

Assistant

> *(progress note)* I'll bring the dialog record up to date through this message, render it as a PDF, and push both files to your repo.

> *(progress note)* While rereading the existing dialog file, I found it puts in my mouth several progress messages I never sent (in sections 5 and 8). Since this is a graded record, I'm rewriting it to match the actual conversation and extending it through today.

The assistant then produced this document and its PDF version, `docs/ai-assistant-dialog.pdf`. Its closing reply to this request is not included, because it was written after the file was generated.